Caracterización del Hombro — ADANISS 2R

Identificación de la planta ampliada por respuesta a pulso

Servomecanismos (2017288) — Universidad Nacional de Colombia
Prof. Victor Hugo Grisales Palacio

June 19, 2026

Abstract

Este documento describe la caracterización experimental del eje del **hombro** del brazo robótico ADANISS 2R, una articulación rotacional accionada por un motorreductor DC con realimentación por encoder de cuadratura. Se detallan los fundamentos conceptuales (decodificación de cuadratura, cálculo del CPR, modelo de la planta), la metodología de medición (posición de caracterización, rigidización del segundo eslabón, guías visuales), el procesamiento de datos en MATLAB y los resultados obtenidos: ganancia de la planta K_{amp}, constante de tiempo τ y zona muerta (*deadband*) en ambos sentidos de giro. Se incluye el desarrollo matemático que justifica cada decisión, en particular la elección de la ventana angular de caracterización a partir de la dinámica del péndulo invertido.

Contents

1 Introducción y objetivo **2**

2 Montaje experimental **2**
2.1 Descripción general . . . 2
2.2 Circuito y conexiones . . . 3
2.3 Rigidización del segundo eslabón con abrazaderas . . . 5
2.4 Guías visuales para la posición de arranque . . . 5

3 Componentes y hardware **5**

4 Conceptos de medición angular **5**
4.1 Encoder de cuadratura y detección de dirección . . . 5
4.2 Cálculo del CPR: 11 PPR× relación de reducción . . . 6

5 Modelo de la planta **6**
5.1 Deducción del modelo de primer orden con integrador . . . 6
5.2 Función de transferencia . . . 7
5.3 La zona muerta (*deadband*) . . . 7

6 Elección de la posición de caracterización **7**
6.1 El par de gravedad y la no linealidad . . . 8
6.2 Por qué la vertical y no otra posición . . . 8
6.3 Por qué un rango tan pequeño (~ 0 a $30°$) . . . 8
6.4 Truncamiento en el procesamiento . . . 8

7 Datos de la zona no lineal: para qué sirven **8**

8 Firmware del ESP32 **9**
8.1 Firmware de caracterización por pulso 9
8.2 Parámetros que se cambian por ensayo 10

9 Procesamiento en MATLAB **10**
9.1 Por qué MATLAB ayuda en la caracterización 10
9.2 `ajuste_planta.m` — ajuste de un archivo 10
9.3 `ajuste_planta_lote.m` — procesamiento de toda la carpeta 10
9.4 `diseno_kp.m` — diseño del controlador 10
9.5 La recta velocidad–PWM 11

10 Resultados **11**

11 Preguntas resueltas durante el desarrollo **12**

12 Resumen y próximos pasos **13**

---

1 Introducción y objetivo

El objetivo de esta fase es obtener un **modelo dinámico** del eje del hombro que permita, posteriormente, diseñar un controlador de posición (control proporcional como primer objetivo) y eventualmente ejecutar una trayectoria de rango amplio (un trébol estilizado) con los dos grados de libertad del brazo.

La estrategia es la **identificación de la planta ampliada** (conjunto driver + motor + eslabón) mediante su **respuesta a pulso** en lazo abierto: se aplica un escalón de PWM de amplitud y duración conocidas, se registra la respuesta angular, y se ajusta un modelo paramétrico a los datos. De ese ajuste se extraen los parámetros que describen el sistema.

Nota metodológica. Siguiendo la recomendación del profesor, no se confía en los valores nominales del fabricante (relación de reducción, ganancias): toda incertidumbre se absorbe empíricamente en la ganancia K_{amp} durante la identificación. Lo único que se calibra a partir de la geometría es la conversión de cuentas del encoder a grados (CPR), y aún esa se verifica de forma experimental.

2 Montaje experimental

2.1 Descripción general

El brazo está montado en **configuración de péndulo invertido**: el pivote del hombro está en la base y el eslabón apunta hacia arriba. La vertical (eslabón apuntando al cenit) se define como el ángulo $\theta = 0°$.



Figure 1: Montaje general del brazo ADANISS 2R en configuración de péndulo invertido.

Qué mostrar en la Imagen 1 (montaje general):

- El brazo completo montado en su base, con el eslabón del hombro apuntando hacia arriba (vertical).
- El **segundo eslabón (codo) inmovilizado con abrazaderas** (ver §2.3), bien visible.
- La **cinta o marca visual** en la base que sirve de referencia de la posición de arranque (ver §2.4).
- El rango físico de movimiento (idealmente con el brazo en vertical y, si se puede, una marca de los costados a $\pm 85°$).
- La fuente de alimentación y la conexión USB al computador.

2.2 Circuito y conexiones

El sistema electrónico se compone de: microcontrolador ESP32, driver de potencia tipo puente H, motorreductor DC con encoder, y sensor de corriente.

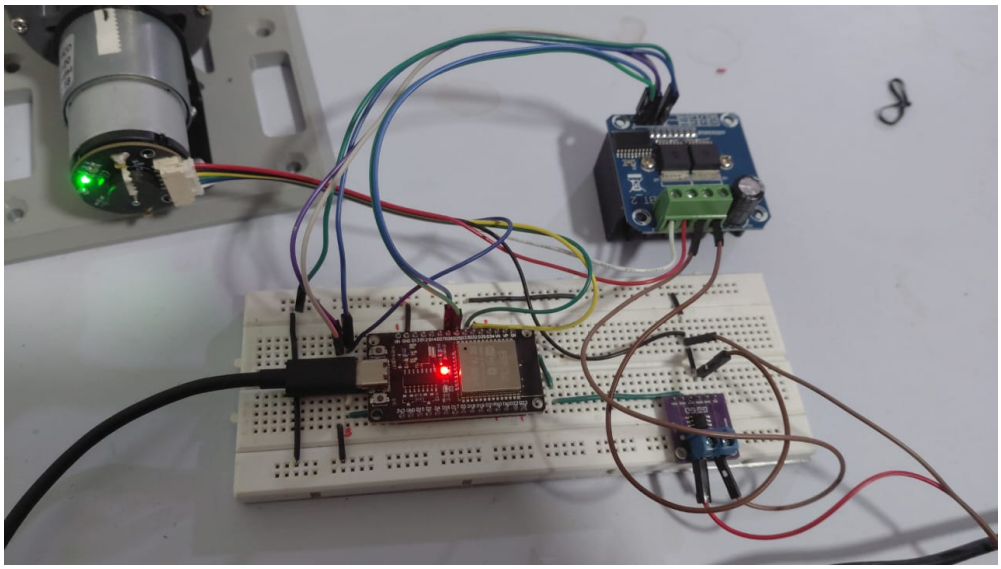


Figure 2: Conexión del ESP32, el driver IBT-2 (BTS7960), el sensor de corriente MAX471 y el motor.

Qué mostrar en la Imagen 2 (circuito):

- El ESP32 con los cables hacia el driver claramente etiquetados.
- El driver IBT-2 con sus terminales de potencia (motor y alimentación) y de lógica.
- El sensor de corriente MAX471 intercalado en la línea positiva de potencia.
- Los cuatro cables del encoder (VCC, GND, A, B) llegando al ESP32.
- El punto de **GND común** entre fuente, ESP32, driver y sensor.

Señal	**Origen**	**Destino (ESP32 / driver)**	**Notas**
RPWM	ESP32 GPIO 25	IBT-2 RPWM	PWM sentido A
LPWM	ESP32 GPIO 26	IBT-2 LPWM	PWM sentido B
R_EN, L_EN	IBT-2	3.3 V (siempre HIGH)	habilitación del puente
R_IS, L_IS	IBT-2	sin conectar (flotante)	evita daño del driver
Encoder A	motor (amarillo)	ESP32 GPIO 32 (INPUT_PULLUP)	canal A
Encoder B	motor (verde)	ESP32 GPIO 33 (INPUT_PULLUP)	canal B
Encoder VCC	motor (azul)	3.3 V	**no** 5 V
Encoder GND	motor (negro)	GND común	
Corriente	MAX471 OUT	ESP32 GPIO 34 (ADC, 11 dB)	
Potencia M+ / M−	IBT-2	terminales del motor	rojo / blanco
Alimentación	fuente 12 V	IBT-2 B+ (vía MAX471) / B−	GND común

Table 1: Conexiones eléctricas del eje del hombro.

Tabla de conexiones.

Precauciones de conexión (aprendidas en la puesta en marcha).

- Los pines `R_IS`/`L_IS` del driver deben quedar **flotantes**; conectarlos sin resistencias en serie dañó un driver previo.
- `R_EN` y `L_EN` deben estar siempre en alto, o el puente no opera.
- El **GND debe ser común** entre la fuente, el ESP32, el driver y el sensor; de lo contrario las señales no tienen referencia (falla silenciosa).
- El encoder se alimenta a 3.3 V (tiene resistencias *pull-up* internas); 5 V dañaría los GPIO del ESP32.

2.3 Rigidización del segundo eslabón con abrazaderas

Durante la caracterización del hombro, el **segundo eslabón (codo) se inmoviliza rígidamente con abrazaderas**. Esto es importante por una razón dinámica:

> En un brazo de dos grados de libertad, la inercia y el par de gravedad que siente el hombro **dependen de la posición del codo**. Si el codo se moviera (o colgara libremente), la planta del hombro cambiaría durante el ensayo y los parámetros identificados no corresponderían a una configuración definida.

Al fijar el codo en una posición conocida y rígida, se garantiza que:

1. La planta del hombro es **invariante** durante todo el ensayo (un solo grado de libertad activo).
2. Los parámetros K_{amp} y τ obtenidos corresponden a esa configuración específica, bien definida y reproducible.
3. Se elimina el acople dinámico entre eslabones, que de otro modo introduciría términos cruzados en las ecuaciones de movimiento.

Esto reduce el problema de identificación de un sistema acoplado de 2 GDL a un sistema de 1 GDL, mucho más simple de modelar. El acople dinámico (cómo cambia K_{amp} del hombro con el ángulo del codo) se deja como refinamiento futuro.

2.4 Guías visuales para la posición de arranque

La caracterización exige que todas las repeticiones de un mismo ensayo **arranquen del mismo punto angular**. Para ello se usan **guías visuales** (cinta o marca) en la base que indican la posición de referencia.

La razón es que el par de gravedad depende del ángulo (§6); si dos repeticiones arrancaran de ángulos distintos, tendrían condiciones de gravedad distintas al inicio y sus respuestas no serían comparables. No se requiere que la marca coincida con la vertical *exacta* del universo: lo que importa es que sea **el mismo punto en todas las repeticiones**. Una tolerancia de $\pm 5^\circ$ es suficiente, porque al promediar varias repeticiones cualquier pequeña variación se atenúa.

3 Componentes y hardware

Componente	Descripción
Microcontrolador	ESP32 DevKitC V4 (Arduino IDE, Core v3.x)
Driver de potencia	IBT-2 (puente H BTS7960)
Sensor de corriente	MAX471 (módulo, rango 3 A)
Motorreductor	DC con reducción y encoder Hall de cuadratura
Encoder	Hall, **11 PPR** en el eje del motor
Alimentación	fuente DC (potencia) + USB (lógica)

Table 2: Componentes del eje del hombro.

4 Conceptos de medición angular

4.1 Encoder de cuadratura y detección de dirección

El encoder entrega dos señales digitales, **canal A** y **canal B**, desfasadas 90° eléctricos entre sí (de ahí el nombre *cuadratura*). Este desfase es lo que permite conocer no solo *cuánto* gira el eje, sino **en qué dirección**.

Principio. Cuando el eje gira en un sentido, el canal A se adelanta al B; en el sentido contrario, el B se adelanta al A. Leyendo el estado de un canal en el flanco del otro se determina el sentido:

$$\text{En flanco de subida de A:} \quad \begin{cases} B = 0 \;\Rightarrow\; \text{sentido } + \text{ (incrementa)} \\ B = 1 \;\Rightarrow\; \text{sentido } - \text{ (decrementa)} \end{cases} \tag{1}$$

Niveles de decodificación (x1, x2, x4). Según cuántos flancos de las señales se cuenten, la decodificación puede ser:

- **x1**: se cuenta un solo tipo de flanco de un canal (p. ej. subida de A). Resolución base.
- **x2**: se cuentan subida y bajada de un canal. Doble resolución.
- **x4**: se cuentan los cuatro flancos (subida y bajada de A y de B). **Cuádruple resolución**.

La decodificación **x4** cuadruplica la resolución angular porque registra los cuatro cambios de estado por ciclo de cuadratura, y es la que mejor aprovecha la información de dirección. En la implementación final se emplea una rutina de interrupción que, en el flanco de A, lee B para decidir el signo; el sentido se ajustó (invirtiendo el signo) para que el avance "hacia adelante" resulte positivo.

Lección de cableado. Un canal del encoder debe conectarse a un GPIO que admita *pull-up*. Los pines de solo entrada del ESP32 (GPIO 34, 35, 36, 39) **no** tienen *pull-up* interno; conectar un canal allí hace que la señal "flote" y la decodificación de dirección falle silenciosamente. Por eso ambos canales se ubicaron en GPIO 32 y 33 (con `INPUT_PULLUP`).

4.2 Cálculo del CPR: 11 PPR× relación de reducción

El encoder está montado en el **eje del motor**, *antes* de la caja reductora. Entrega 11 pulsos por revolución del eje del motor (PPR). Pero lo que nos interesa medir es el ángulo del **eje de salida** (el brazo), que gira mucho más lento debido a la reducción.

Por cada revolución del eje de salida, el eje del motor da N revoluciones, donde N es la relación de reducción. Por tanto, las cuentas por revolución del **eje de salida** (CPR) son:

$$\boxed{\text{CPR} = \text{PPR} \times N \times (\text{factor de decodificación})} \tag{2}$$

Para el motor del hombro, con PPR = 11 y relación de reducción N, la CPR del eje de salida se calcula multiplicando esos dos factores (y el factor de decodificación, 1 en x1 o 4 en x4). Este valor se **verifica empíricamente**: se gira el eje del encoder exactamente una vuelta y se confirma que el contador incrementa en 11 (lo que valida el PPR), y se contrasta una vuelta completa del eje de salida contra el conteo esperado.

La conversión de cuentas a grados es entonces:

$$\theta\,[^\circ] = \text{cuentas} \times \frac{360^\circ}{\text{CPR}} \tag{3}$$

Por qué importa la verificación empírica. Un error en la CPR escala *todos* los ángulos medidos por un factor constante, lo que falsearía directamente K_{amp}. En una iteración previa se usó una CPR equivocada (heredada de una suposición de reducción incorrecta) y los ángulos quedaban mal escalados; recalcular con PPR × N y verificar a mano lo corrigió.

5 Modelo de la planta

5.1 Deducción del modelo de primer orden con integrador

La planta ampliada (driver + motor + eslabón) se modela, para PWM constante y lejos de saturaciones, como un sistema donde el PWM fija una **velocidad de régimen** y la posición es la integral de esa velocidad.

Partiendo de la dinámica del motor DC, la velocidad angular $\omega(t)$ ante un escalón de entrada u (PWM) responde como un sistema de primer orden:

$$\tau\dot{\omega}(t) + \omega(t) = K_{amp}\,u \quad \Rightarrow \quad \omega(t) = K_{amp}\,u\left(1 - e^{-t/\tau}\right) \tag{4}$$

donde:

- K_{amp} es la **ganancia** de la planta [(°/s)/PWM]: velocidad de régimen por unidad de PWM.
- τ es la **constante de tiempo** [s]: mide la rapidez con que el sistema alcanza su velocidad de régimen (debida a la inercia mecánica y la dinámica eléctrica).

La **posición** es la integral de la velocidad:

$$\theta(t) = \int_0^t \omega(s)\,ds = K_{amp}\,u\left[t - \tau\left(1 - e^{-t/\tau}\right)\right] \tag{5}$$

Definiendo la velocidad de régimen $G = K_{amp}\,u$ (la pendiente final de θ vs t), el modelo que se ajusta a los datos es:

$$\boxed{\theta(t) = G\left[t - \tau\left(1 - e^{-t/\tau}\right)\right], \qquad G = K_{amp}\,A} \tag{6}$$

donde A es la amplitud del pulso de PWM.

5.2 Función de transferencia

En el dominio de Laplace, la planta ampliada (de PWM a posición) es:

$$P(s) = \frac{\Theta(s)}{U(s)} = \frac{K_{amp}}{s\,(\tau s + 1)} \tag{7}$$

El término $\dfrac{1}{s}$ es el **integrador** (la posición integra la velocidad) y $\dfrac{1}{\tau s + 1}$ es el polo de primer orden de la dinámica de velocidad. Este integrador implica que, en lazo abierto, la planta *no* mantiene una posición: ante PWM constante, el eje gira indefinidamente. Solo al cerrar el lazo con un controlador aparece la regulación de posición.

5.3 La zona muerta (*deadband*)

El modelo lineal (6) es válido *una vez que el motor se mueve*. Pero por debajo de cierto PWM, la **fricción estática** impide cualquier movimiento: toda la fuerza del motor se consume venciendo fricción. Ese umbral es la **zona muerta** o *deadband*, u_0.

La relación corregida entre velocidad de régimen y PWM es entonces afín, no proporcional:

$$\boxed{G(u) = K_{amp}\,(u - u_0)} \tag{8}$$

La pendiente de esta recta es el K_{amp} "real" del sistema, y su corte con el eje horizontal es la zona muerta u_0. Determinar ambos requiere ensayar a varias amplitudes de PWM y ajustar la recta (§9.5).

6 Elección de la posición de caracterización

Una de las decisiones centrales del experimento es **dónde** y en **qué rango** se caracteriza. La respuesta surge de la dinámica del péndulo invertido.

6.1 El par de gravedad y la no linealidad

Para un eslabón de masa m, con centro de masa a distancia L del pivote, a un ángulo θ medido desde la vertical, el **par gravitacional** es:

$$\tau_g(\theta) = m\, g\, L\, \sin\theta \tag{9}$$

Este término es **no lineal** en θ (por el $\sin\theta$) y es la fuente principal de no linealidad del sistema. Su valor cambia drásticamente con el ángulo:

θ	$0°$	$10°$	$30°$	$45°$	$90°$
$\sin\theta$	0	0.17	0.50	0.71	1.00

6.2 Por qué la vertical y no otra posición

La ecuación (9) muestra que cerca de la vertical ($\theta \approx 0$), el par de gravedad es **mínimo** y, además, varía lentamente. El modelo lineal (6) asume velocidad de régimen *constante* para PWM constante; esto solo se cumple si el par perturbador (la gravedad) es aproximadamente constante durante el ensayo.

Si se caracterizara con el brazo elongado (horizontal, $\theta \approx 90°$): el par de gravedad sería máximo ($\sin 90° = 1$) y, peor aún, cambiaría rápidamente con cada grado de movimiento. La velocidad **no** sería constante: el motor aceleraría o frenaría según la gravedad lo ayude o lo estorbe. El modelo de primer orden *no* ajustaría, y los parámetros serían inválidos.

Conclusión: se caracteriza **cerca de la vertical** porque es la única región donde la gravedad es lo bastante pequeña y estable como para que la hipótesis de velocidad constante (y por tanto el modelo lineal) sea válida.

6.3 Por qué un rango tan pequeño (~ 0 a $30°$)

El rango útil se limita a aproximadamente los primeros $30°$ desde la vertical. La justificación es cuantitativa:

- Entre $0°$ y $30°$, $\sin\theta$ pasa de 0 a 0.5: la gravedad existe pero crece de forma suave, y su efecto sobre la velocidad es pequeño. El modelo lineal sigue siendo una buena aproximación local.
- Más allá de $\sim 40°$, la gravedad supera la fricción estática y el brazo **cae solo** sin necesidad de PWM; la velocidad deja de ser constante y el modelo se rompe.
- El rango mecánico total ($\sim 86°$ hasta el costado) *no* es todo utilizable para identificación lineal: solo la ventana cercana a la vertical lo es.

Aunque $30°$ parezca poco, es suficiente: a amplitudes de PWM bajas el brazo se mueve despacio y entrega **muchas muestras** dentro de esa ventana, lo que da una excelente estimación de la velocidad de régimen G. La calidad del ajuste ($R^2 > 0.99$) confirma que la ventana es adecuada.

6.4 Truncamiento en el procesamiento

Por seguridad y para garantizar la validez del modelo, el ajuste se restringe a $|\theta| \leq 30°$. Incluso si una corrida recorre más ángulo, el script de MATLAB **recorta** los datos a esa ventana antes de ajustar, descartando la zona donde la gravedad domina.

7 Datos de la zona no lineal: para qué sirven

Además de la caracterización lineal cerca de la vertical, se toman datos en la **zona no lineal** (rango amplio, lejos de la vertical). Estos datos **no** se usan para el ajuste lineal; tienen otro propósito:

1. **Demostrar la no linealidad:** un barrido completo muestra que la velocidad *no* es constante cuando la gravedad domina (acelera al caer hacia la vertical, frena al alejarse). Es la evidencia experimental de por qué el modelo lineal se limita a la ventana pequeña.
2. **Calibrar la compensación de gravedad:** la trayectoria de rango amplio (el trébol) operará lejos de la vertical, donde la gravedad es fuerte. Conocer cuánto PWM se necesita para vencer la gravedad en función del ángulo permite diseñar un **término de prealimentación** (*feedforward*) proporcional a $\sin\theta$:

$$u_{ff}(\theta) = k_g \sin\theta \tag{10}$$

   que cancele el par gravitacional (9) y permita que el control lineal funcione en todo el rango.
3. **Mapear los límites operativos:** identificar el PWM mínimo para levantar el brazo desde posiciones desfavorables (cercanas a la horizontal), información necesaria para no exigir al sistema más de lo que puede entregar.

8 Firmware del ESP32

8.1 Firmware de caracterización por pulso

Este programa genera el escalón de PWM y registra la respuesta. Su salida es un CSV con columnas `tiempo_s, angulo_grados, pwm`. Se dispara enviando el carácter `g` por el puerto serie. La secuencia de cada corrida es: una fase de reposo (*baseline*), el pulso de PWM de amplitud A, y una fase de inercia (*coast*) tras apagar el motor.

```
// Modelo de salida CSV: tiempo_s, angulo_grados, pwm
// Disparo: enviar 'g' por serial.
volatile long encCount = 0;
const float CPR = /* 11 PPR * reduccion */;   // verificado empiricamente

// ISR en flanco de subida de A; lee B para el signo (deteccion de direccion)
void IRAM_ATTR isrA() {
  if (digitalRead(ENC_B) == LOW) encCount++;
  else                           encCount--;
}

float anguloDeg() {                  // conversion cuentas -> grados
  long c; noInterrupts(); c = encCount; interrupts();
  return (float)c * 360.0 / CPR;
}

void runPulse() {
  noInterrupts(); encCount = 0; interrupts();   // cero al inicio (angulo relativo)
  unsigned long t0 = millis();
  while ( /* dentro de baseline + pulso + coast */ ) {
    unsigned long now = millis() - t0;
    int pwmCmd = 0;
    if (now < BASELINE_MS)                  pwmCmd = 0;         // reposo
    else if (now < BASELINE_MS + PULSE_MS) pwmCmd = AMPLITUD; // escalon
    else                                    pwmCmd = 0;         // inercia
    if (fabs(anguloDeg()) > ANGLE_LIMIT_DEG) pwmCmd = 0;       // corte de seguridad
    motorPWM(pwmCmd, DIRECCION);
    // ... muestreo cada SAMPLE_MS e impresion CSV ...
  }
  motorStop();
}
```

Listing 1: Núcleo del firmware de caracterización por pulso (ESP32).

Qué hace y por qué.

- **Cero al inicio de cada pulso:** se mide el ángulo *relativo* desde el arranque, por lo que la referencia absoluta es irrelevante para el ajuste.
- **Corte de seguridad por ángulo:** si el brazo supera el límite, se apaga el PWM, evitando que entre en la zona donde la gravedad lo descontrola.
- **Muestreo periódico ($\sim$ cada 5 ms):** suficiente para resolver la dinámica (la constante de tiempo es de orden 10^{-1} s).

8.2 Parámetros que se cambian por ensayo

Solo dos variables cambian entre corridas: la **amplitud** del pulso (A) y el **sentido** de giro. El resto (tiempos de fase, muestreo, límite de ángulo, CPR) permanece fijo.

9 Procesamiento en MATLAB

9.1 Por qué MATLAB ayuda en la caracterización

Los datos crudos son una nube de puntos (ángulo vs tiempo). MATLAB realiza el **ajuste por mínimos cuadrados** del modelo (6) a esos puntos, encontrando los valores de G y τ que mejor reproducen la curva medida. Esto convierte datos experimentales en parámetros del modelo, con una métrica objetiva de calidad (R^2). Hacerlo a mano sería inviable; el ajuste no lineal requiere optimización iterativa.

9.2 `ajuste_planta.m` — ajuste de un archivo

Procesa un único CSV: aísla la ventana del pulso, la recorta a $|\theta| \leq 30°$, ajusta el modelo (6) con `lsqcurvefit`, y entrega K_{amp}, τ y la gráfica del ajuste con su R^2.

9.3 `ajuste_planta_lote.m` — procesamiento de toda la carpeta

Versión por lotes: lee **todos** los CSV de zona lineal de una carpeta, ajusta cada uno, **promedia por amplitud** y por sentido, construye la **recta velocidad–PWM** (8) y entrega K_{amp}, la zona muerta y τ representativo, además de las gráficas y archivos de resumen. Detecta el sentido (*true*/*false*) por el nombre del archivo.

```
% modelo de posicion de 1er orden + integrador
modelo = @(p,t) p(1).*( t - p(2).*(1 - exp(-t./p(2))) );  % p=[G, tau]

% aislar ventana del pulso (pwm != 0) y pasar a angulo relativo en magnitud
idx = find(abs(pwm) > 0);
t   = t(idx(1):idx(end)) - t(idx(1));
ang = abs(ang(idx(1):idx(end)) - ang(idx(1)));

% recortar a la zona limpia (|angulo| <= 30 grados)
m = ang <= ANG_FIT;  t = t(m);  ang = ang(m);

% ajuste por minimos cuadrados -> G y tau
p = lsqcurvefit(modelo, [Gguess, 0.05], t, ang, [0 1e-4], [Inf 5]);
G = p(1);  tau = p(2);  Kamp = G / A;
```

Listing 2: Lógica central del ajuste (modelo y recorte).

9.4 `diseno_kp.m` — diseño del controlador

Con K_{amp} y τ identificados, calcula la ganancia proporcional K_p del controlador de posición según el criterio de desempeño elegido (sin sobrepaso, tiempo de asentamiento dado, etc.).

9.5 La recta velocidad–PWM

Para separar el K_{amp} real de la zona muerta, se ensaya a varias amplitudes ($A = 40, 60, 80, 100$), se obtiene la velocidad de régimen G de cada una, y se ajusta la recta (8). La pendiente es K_{amp} y el corte con el eje es la zona muerta u_0. Ensayar a una sola amplitud no permitiría distinguir ambos efectos.

10 Resultados

Se ensayó el hombro a cuatro amplitudes ($A = 40, 60, 80, 100$) en ambos sentidos de giro (*true*: en el sentido en que la gravedad asiste; *false*: levantando), con tres repeticiones por condición. Todos los ajustes individuales alcanzaron $R^2 > 0.99$.

Sentido	$K_{amp,\text{real}}$ [(°/s)/PWM]	**Zona muerta** [PWM]	τ [s]	R^2 recta
true (asistido)	0.697	16.3	0.139	0.966
false (levanta)	0.675	7.9	0.131	0.976

Table 3: Parámetros identificados de la planta del hombro.

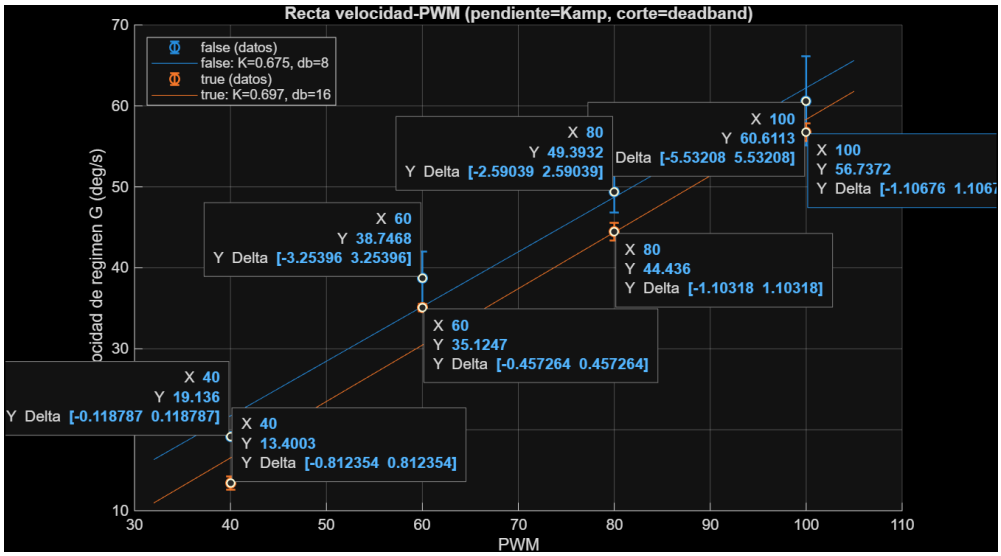


Figure 3: Velocidad de régimen G vs amplitud de PWM, para ambos sentidos. La pendiente es K_{amp}; el corte con el eje horizontal es la zona muerta.

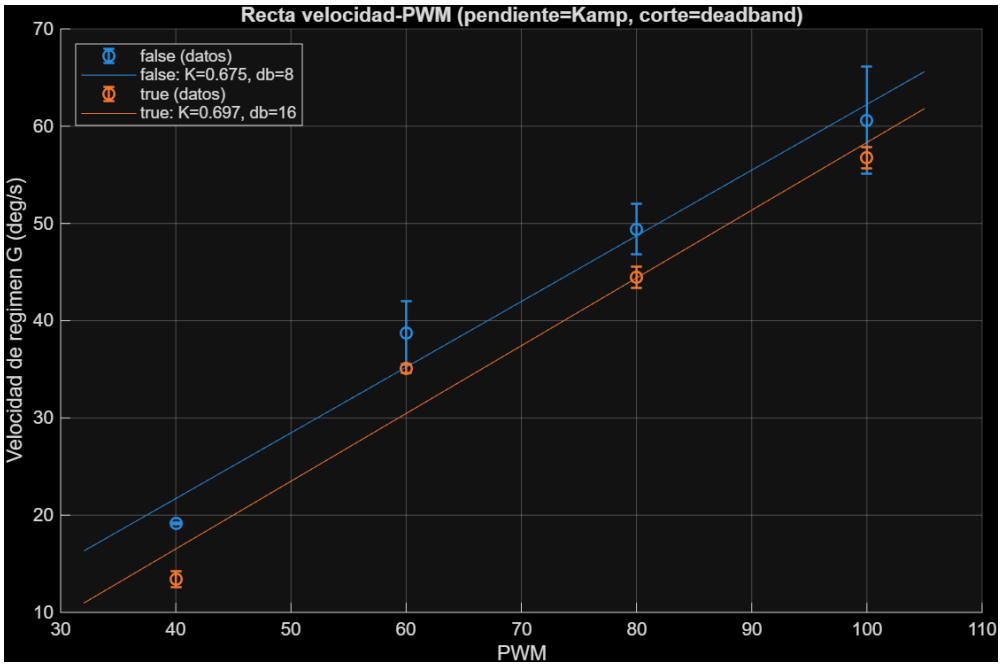


Figure 4: Velocidad de régimen G vs amplitud de PWM, para ambos sentidos. La pendiente es K_{amp}; el corte con el eje horizontal es la zona muerta.

Interpretación.

- Las pendientes (K_{amp}) en ambos sentidos son casi idénticas (0.70 vs 0.68): la ganancia del motor por unidad de PWM es consistente, como se espera de una planta bien identificada.
- La diferencia está en la **zona muerta** (16 en *true* vs 8 en *false*). Esta asimetría es física: al levantar (*false*), la interacción de la gravedad con la fricción estática en el arranque difiere de la del sentido asistido, por lo que el umbral de arranque cambia.
- Un valor representativo para el diseño es $K_{amp} \approx 0.69$ $(°/s)$/PWM y $\tau \approx 0.13\,\text{s}$, tratando la asimetría de zona muerta como un efecto a compensar en el firmware del control.

11 Preguntas resueltas durante el desarrollo

A lo largo del trabajo surgieron varias dudas cuya resolución consolidó la metodología. Se documentan aquí:

1. **¿Las repeticiones de un mismo ensayo van en un archivo o en archivos separados?** En **archivos separados**, uno por repetición. Así cada uno se ajusta por su cuenta y se obtienen varios pares (K_{amp}, τ) que luego se promedian, lo que da robustez y permite ver la variabilidad entre repeticiones.
2. **¿Qué tan exacta debe ser la posición vertical de arranque?** Basta hacerlo a ojo con tolerancia $\pm 5°$, siempre que *todas* las repeticiones arranquen del *mismo* punto. El ajuste usa ángulo relativo, así que la referencia absoluta no entra en K_{amp} ni τ; lo que importa es la consistencia entre repeticiones, asegurada con guías visuales.
3. **¿Qué significan subamortiguado, crítico y sobreamortiguado, y cuándo aparecen?** Aparecen **solo en lazo cerrado** (con el controlador). La planta sola es un integrador (7) que no oscila ni regula posición. El amortiguamiento es una propiedad de diseño que se elige mediante K_p.
4. **¿Por qué se mide del vertical hasta $\sim 30°$ y no en otro rango?** Porque es la ventana donde el par de gravedad (9) es pequeño y estable, condición necesaria para que el modelo lineal sea válido (§6).
5. **¿Qué hace cada script de MATLAB?** `ajuste_planta.m` ajusta un archivo; `ajuste_planta_lote.m` procesa toda la carpeta, promedia y arma la recta; `diseno_kp.m` diseña el controlador.

6. **¿La "calibración" está en el código de caracterización?** No. La calibración del encoder (CPR) es un paso previo, fijo como constante (2). El código de caracterización solo aplica el pulso y mide. El ajuste de parámetros ocurre después, en MATLAB. La referencia de cero absoluto (*homing*) es un tercer paso, propio del control, no de la caracterización.
7. **¿La recta velocidad–PWM con zona muerta negativa es válida?** No: una zona muerta negativa carece de sentido físico y aparecía por extrapolar con pocas amplitudes. Al completar las cuatro amplitudes en ambos sentidos, la zona muerta resultó positiva y coherente (Tabla 3).

12 Resumen y próximos pasos

Logrado.

- Puesta en marcha del hardware del hombro (motor, driver, encoder, sensor).
- Decodificación de cuadratura con detección de dirección y CPR verificada empíricamente.
- Identificación de la planta en ambos sentidos: $K_{amp} \approx 0.69$, $\tau \approx 0.13\,\text{s}$, zona muerta 8–16 PWM.
- Datos de zona no lineal para demostración y futura compensación de gravedad.

Pendiente.

1. Diseñar K_p con los parámetros identificados (`diseno_kp.m`).
2. Implementar y validar el control proporcional de posición en el ESP32.
3. Añadir prealimentación de gravedad (10) para la trayectoria de rango amplio (trébol).
4. Caracterizar el segundo eje (codo) con el mismo procedimiento.